

typing - Literal

It is a typehint used to indicate that a variable, argument, or return value must be strictly equal to a specific, hardcoded value rather than just a general type (like a string or int)

By using `Literal`, you restrict the valid values an object can take.

For example, if a function takes a `Literal["GET", "POST"]`, type checkers will throw an error if you try to pass any other string like `"PUT"`.

Why it is used?

- **Restricting options:** ideal for limiting function inputs to specific valid states, command, or models
- **Better autocomplete:** Code editors (like VS Code) will give you exact autocomplete suggestions since the valid values are known.
- **Return type narrowing:** You can specify an exact return value based on an input (e.g., if you pass `True`, it specifically returns `bytes`; if `False`, it returns `str`).

Code example:

```
from typing import Literal

# The status can only strictly be these three exact strings
def process_order(status: Literal["pending", "shipped", "delivered"]) -> str:
    return f"Order is {status}"

# This works
process_order("shipped")

# This will trigger a type checker error
process_order("processing")
```